KARADENİZ TEKNİK ÜNİVERSİTESİ — BİLGİSAYAR BİLİMLERİ

Hafta 5 Laboratuvar Föyü

React II: Hook'lar, Form, Durum Paylaşımı — Koltuk Seçimini Ortak Bir Sepete Taşıyın

Doç. Dr. Tolga Berber · https://avesis.ktu.edu.tr/tberber

Tahmini süre: 2 saat
Gereksinimler: Node.js 24 LTS kurulu bir bilgisayar (daha yeni bir güncel sürüm — ör. 26 — de çalışır), bir metin editörü (VS Code önerilir), güncel bir tarayıcı (Chrome/Firefox/Edge), Hafta 4'te tamamladığınız `istemci-react` klasörü — npm paketlerini indirmek için internet bağlantısı gerekir

Öğrenme fazları: 1 • Ön-öğretim | 2 • İçerik | 3 • Katılım | 4 • Değerlendirme | 5 • İzleme

1 • Ön-öğretim — Neden Önemli?

Geçen haftanın laboratuvarında bir koltuğa tıkladığınızda `Koltuk` bileşeni kendi rengini değiştiriyordu, ama bu bilginin `App`'e, bir sepet özetine ya da bir bilet formuna ulaşmasının hiçbir yolu yoktu — her koltuk kendi kapalı kutusunda yaşıyordu. Bugün iki saat sonra aynı koltuk verisiyle, seçili koltukları tek bir ortak yerde toplayacak, sekme başlığını ve bir geri sayımı `useEffect` ile güncelleyecek, gerçek bir formla bilet oluşturacak ve tekrar eden mantığı özel hook'lara taşıyacaksınız — Etkinlik/Bilet Sisteminin ilk gerçek sepet katmanı bugün ortaya çıkacak.

1 • Ön-öğretim — Ön Bilgi

Bu föy Hafta 5 ders notunun dört ünitesine dayanır: Ünite 1 (`useEffect` ve Yaşam Döngüsü), Ünite 2 (Formlar ve Kontrollü Girdi), Ünite 3 (Durum Kaldırma ve Context) ve Ünite 4 (Özel Hook'lar ve Kompozisyon). Hafta 4'ün `istemci-react` projesini — `tipler.ts/veri.ts` dosyalarını, `EtkinlikKarti/ EtkinlikListesi/Bolum/KoltukPlani` bileşenlerini — aynen kullanmaya devam edeceksiniz. Hafta 4 Ünite 3'ün `useState`/değişmezlik ilkesini ve YG II'nin `std::thread`'i başlatıp durdurma disiplinini (temizleme fonksiyonu için köprü) hazır bilgi olarak kullanacaksınız. Sunucu (FastAPI), `fetch`, gerçek bir API ve React Router bu hafta da yoktur — veri hâlâ `veri.ts`'in sabit dizilerinden gelir (Hf.6–7'nin konusudur).

Föyü şöyle kullanın: her adımda önce *Ne yapacağız?* kutusundaki görevi kendi `istemci-react` projenize **elle yazarak** tamamlayın, sonra editörünüzde/tarayıcınızda gördüğünüzü *Beklenen sonuç* kutusuyla karşılaştırın. Adımlardaki kod kutuları `kod_ornekleri/hafta05/istemci/` klasöründeki referans dosyalardan gelir — bunlar hedef/"doğru cevap" niteliğindedir; siz bu kodu kopyala-yapıştır yapmak yerine kendi dosyalarınıza yazarsınız, sonucu editörde/tarayıcıda karşılaştırırsınız. Birkaç adımda bir gelen *Kontrol Noktası*, o ana kadar yazdıklarınızı bir arada toparlamanıza yarar.

Bölüm sırası hakkında bir not: `BiletFormu`, `SepetOzeti` ve `GeriSayim` bileşenlerinin hepsi az sonra kuracağınız `SepetContext`'e dayanır — bu yüzden bu föyde Context'i (ders notunun Ünite 3'ü), Formlar bölümünden (Ünite 2) önce kuracaksınız. Kodun her adımda derlenebilir/çalışır kalması için gerçek yazılım geliştirmede de sık yapılan bir şeyi yapıyorsunuz: önce paylaşılan temel (Context), sonra ona dayanan parçalar (form, özet, geri sayım). Her bölüm başlığının yanında

hangi üniteye dayandığını yine de bulacaksınız.

1 `useEffect` ve Yaşam Döngüsü

(*bkz. Ders notu Hafta 5, Ünite 1 — `useEffect` ve Yaşam Döngüsü.*)

Adım 1: Hafta 4 projenizin çalıştığını doğrulayın

Ne yapacağız? `istemci-react` klasörünüzü açacak, `npm run dev` ile geliştirme sunucusunu başlatacak ve tarayıcıda Hafta 4'te bıraktığınız hâliyle (3 etkinlik kartı, 3×4 koltuk ızgarası, koltuklara tıklayınca renk değişimi) çalıştığını doğrulayacaksınız.

Neden? Bugünkü tüm adımlar bu projenin üzerine inşa edilecek — yeni bir proje oluşturmayacaksınız. Devam etmeden önce geçen haftanın kodunun sağlam durduğundan emin olmak, bugün karşılaşacağınız bir hatanın yeni kodunuzdan mı yoksa eski bir sorundan mı geldiğini ayırt etmenizi kolaylaştırır.

Terminal

```
$ cd istemci-react
$ npm run dev
```

Beklenen sonuç: Terminalde Vite'ın başlangıç bildirimi görünür; tarayıcıda `http://localhost:5173` adresinde iki bölüm ("Etkinlikler", "Koltuk Planı") açılır.

Beklenen Çıktı

```
  VITE v8.2.2  ready in 110 ms

  ➜  Local:   http://localhost:5173/
  ➜  Network: use --host to expose
```

İpucu

Geliştirme sunucusunu bugünün geri kalanında açık bırakabilirsiniz — Vite'ın anlık güncelleme (HMR) özelliği her kaydettiğinizde sayfayı otomatik yeniler, `npm run dev`'i tekrar tekrar başlatmanıza gerek yoktur.

Adım 2: `useGeriSayim` fonksiyonunu yazıp izole test edin

Ne yapacağız? `src/hooks` klasörünü oluşturup içine `useGeriSayim.ts` dosyasını ekleyecek; `baslangicSaniye: number` parametresi alan, `useState` ile kalan saniyeyi tutan, `useEffect` içinde `setInterval` ile bu değeri her saniye azaltan ve temizleme fonksiyonuyla `clearInterval` çağıran fonksiyonu yazacaksınız. Ardından sadece test amacıyla, `src/App.tsx`'in en üstüne `import { useGeriSayim } from './hooks/useGeriSayim.ts';` ekleyip `App` fonksiyonunun içine geçici olarak `const { kalanSaniye } = useGeriSayim(10);` satırını ve JSX'e `<p>Test: {kalanSaniye} </p>` satırını ekleyeceksiniz.

Neden? Bu ünitede fonksiyonu henüz "özel hook" olarak adlandırmıyoruz — resmî tanımını Ünite 4'te göreceksiniz; şimdilik yalnızca "state ve efekt kullanan bir fonksiyon" olarak düşünün. `setInterval` her saniye çalışan bir zamanlayıcı kurar; `useEffect`'in döndürdüğü `() => clearInterval(id)` bu zamanlayıcıyı durdurur. Bu satır olmasaydı, bileşen ekrandan kalksa (*unmount*) bile zamanlayıcı arka planda çalışmaya devam ederdi — YG II'de bir

Adım 2 (devamı)

`std::thread`'i `join/detach` etmeden bırakmanın yol açtığı soruna benzer bir durum. Geçici test satırları, bu fonksiyonun `GeriSayim` bileşenine bağlanmadan önce de kendi başına çalıştığını gözünüzle görmenizi sağlar.

```
// baslangicSaniye'den geriye sayar: useState ile kalanSaniye tutulur,
// useEffect içindeki setInterval her saniye setKalanSaniye'yi çağırır.
// Süre bitince (bittiMi) zamanlayıcı YENİDEN kurulmaz; efekt her
// çalıştığında (ve bileşen ekrandan kalktığında) döndürdüğü temizleme
// fonksiyonuyla clearInterval çağrılır — bu olmasaydı bileşen unmount
// olsa bile zamanlayıcı arka planda çalışmaya devam ederdi.
export function useGeriSayim(baslangicSaniye: number) {
  const [kalanSaniye, setKalanSaniye] = useState(baslangicSaniye);
  const bittiMi = kalanSaniye <= 0;

  useEffect(() => {
    if (bittiMi) return;

    const id = setInterval(() => {
      setKalanSaniye((onceki) => (onceki <= 1 ? 0 : onceki - 1));
    }, 1000);

    return () => clearInterval(id);
  }, [bittiMi]);

  return { kalanSaniye, bittiMi };
}
```

Beklenen sonuç: Tarayıcıda sayfanın en üstünde "Test: 10" yazısı görünür ve her saniye bir azalarak "Test: 9", "Test: 8" diye iner; 0'a ulaştığında durur (yeniden başlamaz, çünkü `bittiMi` olduğunda efekt erken `return` yapar).

Ortak Hata / Tuzak

`setInterval`'in döndürdüğü kimliği (`id`) bir değişkende tutmayı ve temizleme fonksiyonunda `clearInterval(id)` ile kullanmayı unutmak en sık yapılan hatadır — bu durumda zamanlayıcı hiçbir zaman durmaz, bileşen kalksa bile arka planda çalışmaya devam eder ve tarayıcı konsoluna "componente ait olmayan bir state güncellemesi" uyarısı basılabilir.

Adım 3: Belge başlığı efektini izole bir sayaçla deneyimleyin

Ne yapacağız? `App.tsx`'teki geçici test satırlarının hemen altına, yine sadece test amacıyla, `const [sayac, setSayac] = useState(0);` satırını, bu sayacı 1 artıran bir düğmeyi (`<button onClick={() => setSayac(sayac + 1)}>Koltuk Seç (test)</button>`) ve `document.title`'ı `sayac`'a göre güncelleyen bir `useEffect(() => { document.title = `(${sayac}) Test`; }, [sayac]);` satırını ekleyeceksiniz.

Neden? Bu, gerçek `app_baslik_efekti` kodunun — birazdan Ünite 3'te kuracağınız `SepetContext`'ten gelen `secilenKoltuklar.length` yerine, şimdilik basit bir sayaç kullanan — izole bir provasıdır. Bağımlılık dizisinin `[sayac]` olması, efektin yalnızca `sayac` bir önceki render'a göre değiştiğinde tekrar çalışacağı anlamına gelir — şimdi bunu bozup gözlemleyeceksiniz.

```
useEffect(() => {
```

Adım 3 (devamı)

```
  document.title = `(${sayac}) Test`;
}, [sayac]);
```

Beklenen sonuç: Düğmeye her tıkladığınızda tarayıcı sekmesinin başlığı "(1) Test", "(2) Test" diye güncellenir.

Dene-Çalıştır

Bağımlılık dizisini geçici olarak `[]` (boş dizi) yapın, sayfayı yenileyip düğmeye birkaç kez tıklayın — sekme başlığının artık güncellenmediğini, ilk hâlinde `(0) Test` donduğunu gözlemleyin. React, bir bağımlılık dizisi vermediğiniz sürece "ne zaman yeniden çalışacağını" sizden öğrenir, kendiliğinden tahmin etmez. Deneyi bitirince bağımlılığı `[sayac]`'a geri döndürün, ardından bu adımdaki tüm geçici test kodunu (sayaç, düğme, efekt, `useGeriSayim` test satırları) `App.tsx`'ten silin — gerçek sürümlerini ilerleyen bölümlerde kuracaksınız.

Kontrol Noktası — devam etmeden doğrulayın

`useGeriSayim` fonksiyonunuz izole testte doğru saydıysa, belge başlığı deneyinde bağımlılık dizisinin `[]` ile donduğunu, `[sayac]` ile güncellendiğini gördüyseniz ve `App.tsx`'i geçici kodlardan temizlediyseniz, `useEffect`'in üç davranışını (dizi yok/boş/dolu) kavramışsınız demektir — artık birden fazla bileşenin paylaşacağı ortak durumu kurabilirsiniz.

2 Durum Kaldırma (Lifting State Up) ve Context

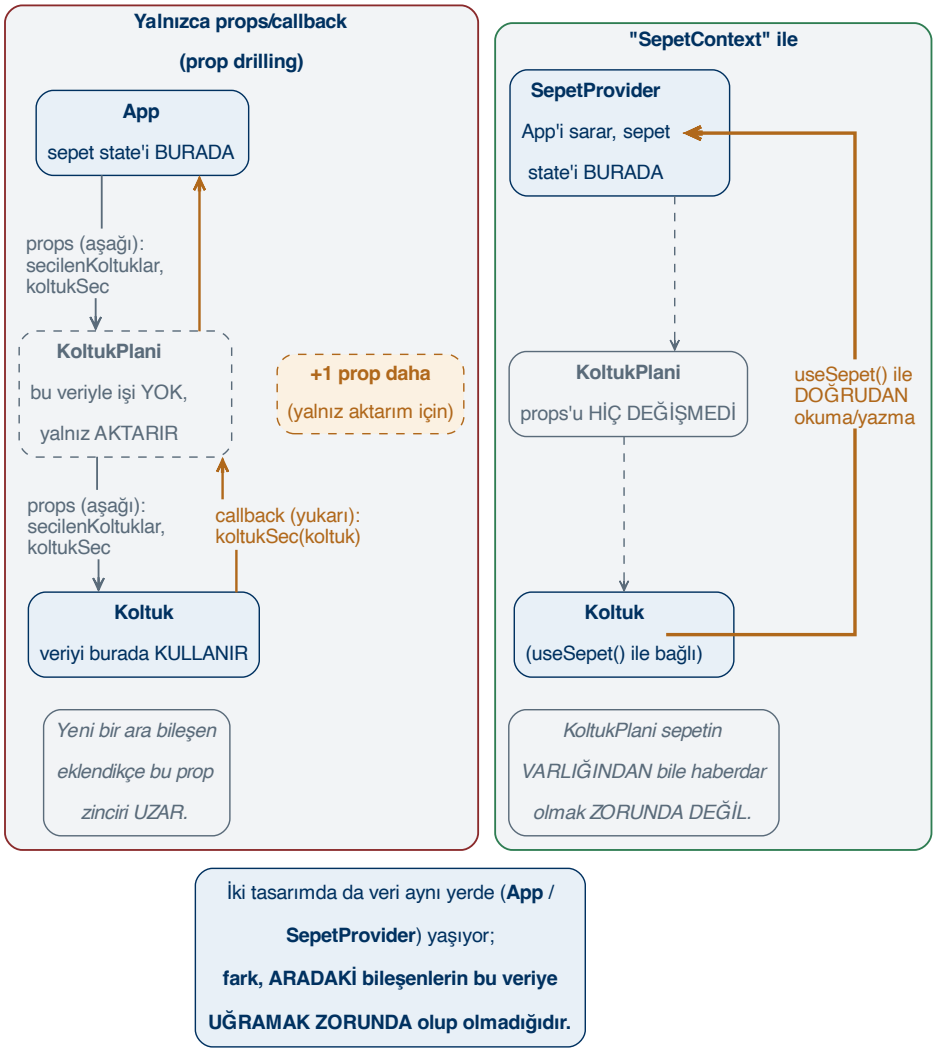
(*bkz. Ders notu Hafta 5, Ünite 3 — Durum Kaldırma ve Context.*)

Adım 4: `SepetContext.tsx` dosyasını yazın

Ne yapacağız? `src/context` klasörünü oluşturup içine `SepetContext.tsx` dosyasını ekleyecek; `secilenKoltuklar`, `toplamTutar`, `koltukSec` ve `sepetiTemizle` alanlarını taşıyan `SepetBaglami` arayüzünü, `createContext<SepetBaglami | null>(null)` ile `SepetContext`'i ve `biletFiyati/children` props'u alan `SepetProvider` bileşenini yazacaksınız.

Neden? Geçen hafta `Koltuk` bileşeni kendi `durum` state'ini tutuyordu, ama `SepetOzeti` (kaç koltuk seçili, toplam tutar ne) ve `BiletFormu` (hangi koltuklar için bilet kesilecek) bu bilgiye ihtiyaç duyuyor — ve bu bilgi `Koltuk`'un içinde hapis. **Durum kaldırma** (*lifting state up*) birden fazla bileşenin paylaşması gereken bir durumu ortak bir üst noktaya taşımaktır; bir **Context** ise bu ortak durumu, ağacın herhangi bir alt noktasındaki bileşenin aradaki bileşenlerin haberi olmadan doğrudan okuyabildiği bir kanaldır. Başlangıç değerinin `null` olması bilinçli bir tercihtir: bir bileşen `SepetProvider`'ın dışında kullanılırsa bunu birazdan bir hatayla ayırt edebileceksiniz.

Adım 4 (devamı)




```ts
// Sepetin dışa açtığı veri ve işlevler: seçili koltuklar, toplam tutar,
// bir koltuğu sepete ekleyip çıkaran ve sepeti boşaltan iki fonksiyon.
export interface SepetBaglami {
  secilenKoltuklar: Koltuk[];
  toplamTutar: number;
  koltukSec: (koltuk: Koltuk) => void;
  sepetiTemizle: () => void;
}
```

```ts
// Başlangıç değeri null: bir bileşen SepetProvider'ın DIŞINDA
// useContext çağırırsa bunu ayırt edebilmek için (bkz. hooks/useSepet.ts).
export const SepetContext = createContext<SepetBaglami | null>(null);
```

```ts
// Seçili koltukları tek bir useState ile tutar; koltukSec aynı koltuk
// zaten sepetteyse çıkarır, değilse ekler (değişmezlik ilkesiyle her
// zaman YENİ bir dizi üretir). toplamTutar her render'da
// secilenKoltuklar'dan yeniden hesaplanır, ayrıca saklanmaz.
export function SepetProvider({ biletFiyati, children }: SepetProviderProps) {
  const [secilenKoltuklar, setSecilenKoltuklar] = useState<Koltuk[]>([]);

  function koltukSec(koltuk: Koltuk) {
    setSecilenKoltuklar((onceki) =>
```

Adım 4 (devamı)

```
    onceki.some((k) => k.id === koltuk.id)
      ? onceki.filter((k) => k.id !== koltuk.id)
      : [...onceki, koltuk],
  );
 }

 function sepetiTemizle() {
  setSecilenKoltuklar([]);
 }

 const deger: SepetBaglami = {
  secilenKoltuklar,
  toplamTutar: secilenKoltuklar.length * biletFiyati,
  koltukSec,
  sepetiTemizle,
 };

 return <SepetContext.Provider value={deger}>{children}</SepetContext.Provider>;
}
```

Beklenen sonuç: Dosyayı kaydettiğinizde editörünüzde hiçbir kırmızı çizgi görünmez — bu Context'i henüz hiçbir bileşen kullanmıyor, tarayıcıda şimdilik hiçbir değişiklik görmeyeceksiniz.

> **İpucu**
>
> Aynı dosyada, sepetteki eylemleri (`EKLE`/`CIKAR`/ `TEMIZLE`) tek bir fonksiyonda toplayan `useReducer` tabanlı bir alternatif (`sepetReducer`) de bulunur — bu haftanın çalışan sepet mantığı sadelik için yukarıdaki `useState`'i kullanmaya devam eder; `sepetReducer` yalnızca dışa aktarılır, `SepetProvider`'a bağlanmaz. Birden çok ilişkili eylem biriktiğinde `useReducer`'ı bir alternatif olarak aklınızda tutun.

Adım 5: `useSepet` özel hook'unu yazıp `throw` davranışını gözlemleyin

Ne yapacağız? `src/hooks/useSepet.ts` dosyasını oluşturup `useContext(SepetContext)`'i saran, `null` dönerse açık bir `Error` fırlatan `useSepet` fonksiyonunu yazacaksınız. Ardından sadece test amacıyla, `App.tsx`'teki `App` fonksiyonunun içine (`SepetProvider` ile henüz sarmadan) `const { toplamTutar } = useSepet();` satırını geçici olarak ekleyip tarayıcı konsolunu (F12) açacaksınız.

Neden? Her bileşenin kendi başına `useContext(SepetContext)` yazıp `null` kontrolü yapması yerine, bu kontrolü tek bir yerde toplamak istiyorsunuz — Ünite 4'te bunun resmî adının "özel hook" olduğunu göreceksiniz. `baglam` `null` dönerse (bileşen `SepetProvider`'ın dışında kullanılmışsa) açık bir `throw`, "`toplamTutar` `undefined`" gibi belirsiz bir hatayla geç fark edilecek bir sorunu hemen, anlaşılır bir mesajla ortaya çıkarır.

```
// useContext(SepetContext)'i sarar; her bileşenin kendi başına
// useContext + null kontrolü yazması yerine yalnızca useSepet() çağırması
// yeter. baglam null dönerse (SepetProvider dışında kullanılmışsa)
// belirsiz bir çalışma zamanı hatası yerine AÇIK, anlaşılır bir hata
// fırlatır.
export function useSepet() {
 const baglam = useContext(SepetContext);
 if (!baglam) {
  throw new Error('useSepet() yalnızca SepetProvider içinde kullanılabilir.');
```

Adım 5 (devamı)

```
  }
  return baglam;
}
```

Terminal

```
$ npm run dev
```

Beklenen sonuç: Tarayıcı beyaz bir hata ekranı gösterir; DevTools konsolunda aşağıdakine benzer bir hata mesajı görünür.

Beklenen Çıktı

```
Uncaught Error: useSepet() yalnızca SepetProvider içinde kullanılabilir.
    at useSepet (useSepet.ts:13:11)
    at App (App.tsx:9:34)
```

Ortak Hata / Tuzak

`useSepet`'in içindeki `if (!baglam) throw new Error(...)` satırını silerseniz, aynı hata yerine `toplamTutar` okunmaya çalışıldığında "Cannot read properties of null" gibi belirsiz bir çalışma zamanı hatası alırsınız — hatanın gerçek nedeni (Provider eksikliği) mesajdan anlaşılmaz. Açık `throw`, yanlış kullanım yerini hemen, net biçimde gösterir.

Beklenen sonuç: Geçici test satırını (`const { toplamTutar } = useSepet();`) `App.tsx`'ten silin — bir sonraki adımda `App`'i `SepetProvider` ile doğru biçimde saracaksınız.

Adım 6: `Koltuk` bileşenini `useSepet` ile güncelleyip `App`'i `SepetProvider` ile sarın

Ne yapacağız? `Koltuk.tsx`'te `useState` içe aktarmasını ve yerel `durum` state'ini kaldıracak, yerine `useSepet` içe aktarıp `const { secilenKoltuklar, koltukSec } = useSepet();` ile Context'ten okuyacaksınız; `durum` artık `koltuk.durum === 'dolu' ? 'dolu' : seciliMi ? 'secili' : 'bos'` biçiminde hesaplanacak, `tikla` fonksiyonu `koltukSec(koltuk)`'u çağıracaktır. Ardından `App.tsx`'te `App` fonksiyonunun döndürdüğü JSX'i `<SepetProvider biletFiyati={etkinlikler[0].biletFiyati}>...</SepetProvider>` ile saracaksınız (`etkinlikler`'i `veri.ts`'ten zaten içe aktarıyordunuz).

Neden? Koltuğun seçili olup olmadığı artık kendi yerel state'inde değil, ortak `SepetContext`'teki `secilenKoltuklar` dizisinde tutulur — `App`'in ya da `KoltukPlani`'nın haberi olmadan bu bileşen doğrudan Context'ten okur. Dikkat edin: `KoltukPlani`'nın `props` tanımına tek bir yeni alan eklemediniz — eğer bunu `props`/callback ile yapsaydınız, `KoltukPlani`'nın props'una — kendisi bu veriyle hiçbir iş yapmadığı hâlde — `secilenKoltuklar`/`koltukSec` gibi alanlar eklemeniz gerekirdi (*prop drilling*); Context bu ara katmanı tamamen ortadan kaldırır.

```
// Koltuğun seçili olup olmadığı artık kendi yerel state'inde değil,
// ortak SepetContext'teki secilenKoltuklar dizisinde tutulur; App/
// KoltukPlani'nın haberi olmadan bu bileşen doğrudan Context'ten okur.
// Tıklama koltukSec'i çağırıp koltuğu sepete ekler/çıkarır; dolu bir
// koltuk (veri.ts'ten gelen kalıcı durum) hiç tıklanamaz.
```

Adım 6 (devamı)

```
const { secilenKoltuklar, koltukSec } = useSepet();
const seciliMi = secilenKoltuklar.some((k) => k.id === koltuk.id);
const durum = koltuk.durum === 'dolu' ? 'dolu' : seciliMi ? 'secili' : 'bos';

function tikla() {
  if (koltuk.durum === 'dolu') return;
  koltukSec(koltuk);
}
```

Beklenen sonuç: Tarayıcıyı yenilediğinizde koltuk planı yine görünür; boş bir koltuğa tıkladığınızda rengi "seçili" durumuna döner, tekrar tıklayınca geri döner — görsel davranış Hafta 4 ile aynı görünür, ama artık bu bilgi `Koltuk`'un dışından da erişilebilir (bir sonraki adımda `SepetOzeti` ile bunu göstereceksiniz). `dolu` işaretli koltuklar (B2, C4) hâlâ tıklanamaz.

> **Dikkat**
>
> `AppIcerik` bileşenini (`SepetProvider`'ın içinde duran bileşeni) Bölüm 4'te kuracaksınız — bir bileşen kendi sağladığı Context'i doğrudan okuyamaz, yalnızca alt'ları okuyabilir. Şimdilik `App` fonksiyonunun kendisinin `SepetProvider`'ı sarmasıyla yetinin; `koltuklar/biletler` state'ini ve belge başlığı efektini son bölümde ekleyeceksiniz.

Kontrol Noktası — devam etmeden doğrulayın

`SepetContext.tsx` ve `useSepet.ts` dosyalarınız hazırsa, `Provider` dışında kullanım denemenizde konsol hatasını gördüyseniz ve `Koltuk` artık `useSepet` ile sepete ekleyip çıkarıyorsa, durum kaldırma ve Context'in temelini kurmuşsunuz demektir — artık bu ortak duruma dayanan bir form yazabilirsiniz.

3 Formlar ve Kontrollü Girdi

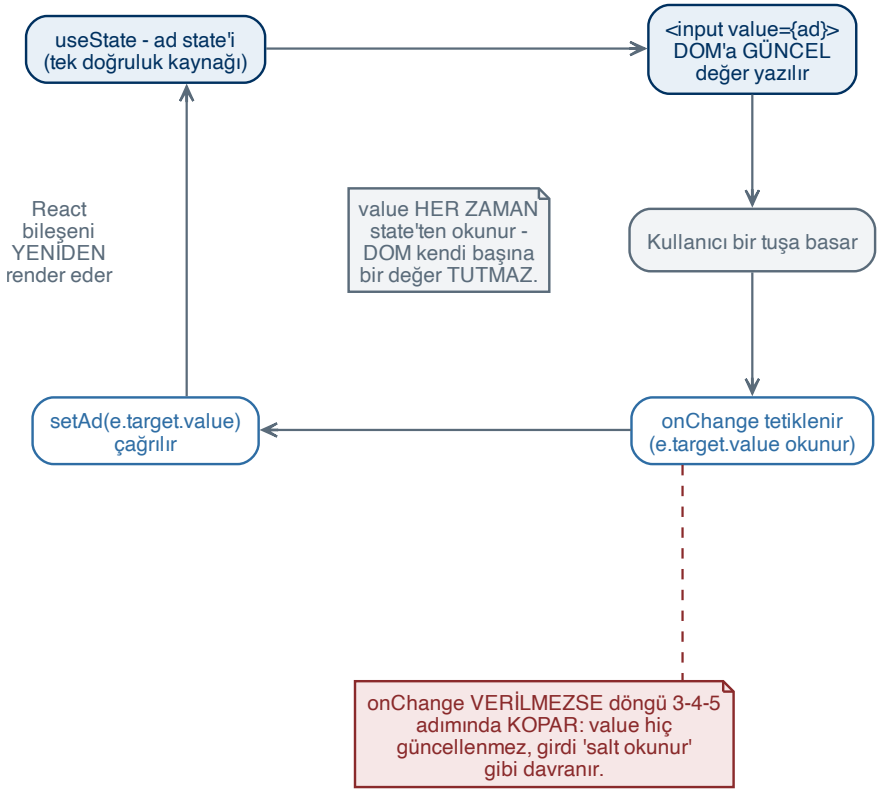
(bkz. Ders notu Hafta 5, Ünite 2 — Formlar ve Kontrollü Girdi.)

Adım 7: `BiletFormu` bileşenini kontrollü girdilerle yazın

Ne yapacağız? `src/bilesenler/BiletFormu.tsx` dosyasını oluşturup `ad/koltukSinifi/kampanyaOnay` alanlarını taşıyan `BiletFormVeri` arayüzünü, tek bir `useState<BiletFormVeri>` çağrısıyla tutulan form state'ini, metin girdisi + `select` + `checkbox` içeren render kısmını ve `onSubmit`'te `e.preventDefault()` ile başlayan, ad boşsa ya da sepet boşsa `hata` state'ini dolduran doğrulama mantığını yazacaksınız.

Neden? Hafta 3'ün bilet formu düz HTML'di — `submit` olayında yalnızca `preventDefault()` çağrılıp konsola bir özet basılıyordu, React'in ise haberi bile yoktu. **Kontrollü girdi** bir form öğesinin değerinin her zaman React state'inden okunmasıdır: `value/onChange` çifti metin girdisi ve `select` için, `checked/onChange` çifti ise `checkbox` için kullanılır — `checkbox`'ta `e.target.value` değil `e.target.checked` okunur. Bir alanı güncellerken diğer alanları korumak için yayma operatörü (`{ ...form, ad: e.target.value }`) kullanılır — Hafta 4'ün değişmezlik ilkesinin form state'ine uygulanışıdır. `onSubmit` işleyicisinin ilk satırının `e.preventDefault()` olması zorunludur — aksi hâlde tarayıcının varsayılan davranışı (sayfayı yeniden yüklemek) devreye girer ve tüm React state'i sıfırlanır.

Adım 7 (devamı)




```tsx
// Formun TÜM alanlarını tek bir tipte gruplar; koltukSinifi sınırlı bir
// seçenek kümesinden (standart/vip) gelir, Hf.2'nin bilet_formu HTML
// yapısının React karşılığıdır.
export interface BiletFormVeri {
  ad: string;
  koltukSinifi: 'standart' | 'vip';
  kampanyaOnay: boolean;
}

const BOS_FORM: BiletFormVeri = {
  ad: '',
  koltukSinifi: 'standart',
  kampanyaOnay: false,
};

interface BiletFormuProps {
  onBiletlerOlustu: (biletler: Bilet[]) => void;
}
```

```tsx
// Tüm alanlar TEK bir useState çağrısında tutulur; bir alanı
// güncellerken diğerlerini KORUMAK için yayma operatörü kullanılır
// (Hf.4'ün değişmezlik ilkesinin form state'ine uygulanışı).
const [form, setForm] = useState<BiletFormVeri>(BOS_FORM);
const [hata, setHata] = useState<string | null>(null);
const { secilenKoltuklar, sepetiTemizle } = useSepet();
```

```tsx
// İlk satır e.preventDefault(): aksi halde tarayıcı VARSAYILAN
// davranışıyla sayfayı yeniden yükler ve tüm React state'i sıfırlanır.
// Sonra iki koşul kontrol edilir (ad boş mu, sepette hiç koltuk yok
// mu); koşullardan biri sağlanmazsa hata state'i doldurulur ve Bilet[]
```

Adım 7 (devamı)

```
// OLUŞTURULMAZ.
function submitEt(e: FormEvent<HTMLFormElement>) {
  e.preventDefault();

  if (form.ad.trim() === '') {
    setHata('Ad alanı boş olamaz.');
    return;
  }
  if (secilenKoltuklar.length === 0) {
    setHata('Sepette en az bir koltuk seçili olmalı.');
    return;
  }

  const satinAlmaZamani = new Date().toISOString();
  const yeniBiletler: Bilet[] = secilenKoltuklar.map((koltuk) => ({
    id: sonrakiBiletId++,
    koltukId: koltuk.id,
    kullaniciAdi: form.ad,
    satinAlmaZamani,
    durum: 'olusturuldu',
  }));

  onBiletlerOlustu(yeniBiletler);
  sepetiTemizle();
  setForm(BOS_FORM);
  setHata(null);
}
```

```
return (
  <form className="bilet-formu" onSubmit={submitEt}>
    <label>
      Ad Soyad
      <input
        type="text"
        value={form.ad}
        onChange={(e) => setForm({ ...form, ad: e.target.value })}
      />
    </label>
    <label>
      Koltuk Sınıfı
      <select
        value={form.koltukSinifi}
        onChange={(e) =>
          setForm({
            ...form,
            koltukSinifi: e.target.value as BiletFormVeri['koltukSinifi'],
          })
        }
      >
        <option value="standart">Standart</option>
        <option value="vip">VIP</option>
      </select>
    </label>
    <label>
      <input
        type="checkbox"
        checked={form.kampanyaOnay}
```

Adım 7 (devamı)

```
      onChange={(e) => setForm({ ...form, kampanyaOnay: e.target.checked })}
    />
    Kampanya duyurularını almak istiyorum
  </label>
  {hata && <p className="hata">{hata}</p>}
  <button type="submit">Bilet Oluştur</button>
 </form>
);
```

Beklenen sonuç: Dosyada hata yok; bu bileşeni henüz `App.tsx`'te kullanmadınız, tarayıcıda hâlâ görünmeyecektir — bir sonraki bölümde `App`'e bağlayacaksınız.

Ortak Hata / Tuzak

`<input value={form.ad} />` yazıp `onChange` vermemek yaygın bir hatadır — React konsola "salt okunur alan" uyarısı basar; `value` her render'da aynı state'ten okunduğundan kullanıcı hiçbir şey yazamaz, tuşuna bastığı harf ekranda görünmez.

Kontrol Noktası — devam etmeden doğrulayın

`BiletFormu.tsx` dosyanız editörde hatasız duruyorsa, `value/onChange` ve `checked/onChange` çiftlerini doğru eşleştirdiyseniz ve doğrulama mantığınız `e.preventDefault()` ile başlıyorsa, kontrollü girdi temelini kurmuşsunuz demektir — artık tekrar eden mantığı özel hook'lara toplayıp her şeyi birleştirebilirsiniz.

4 Özel Hook'lar ve Kompozisyon

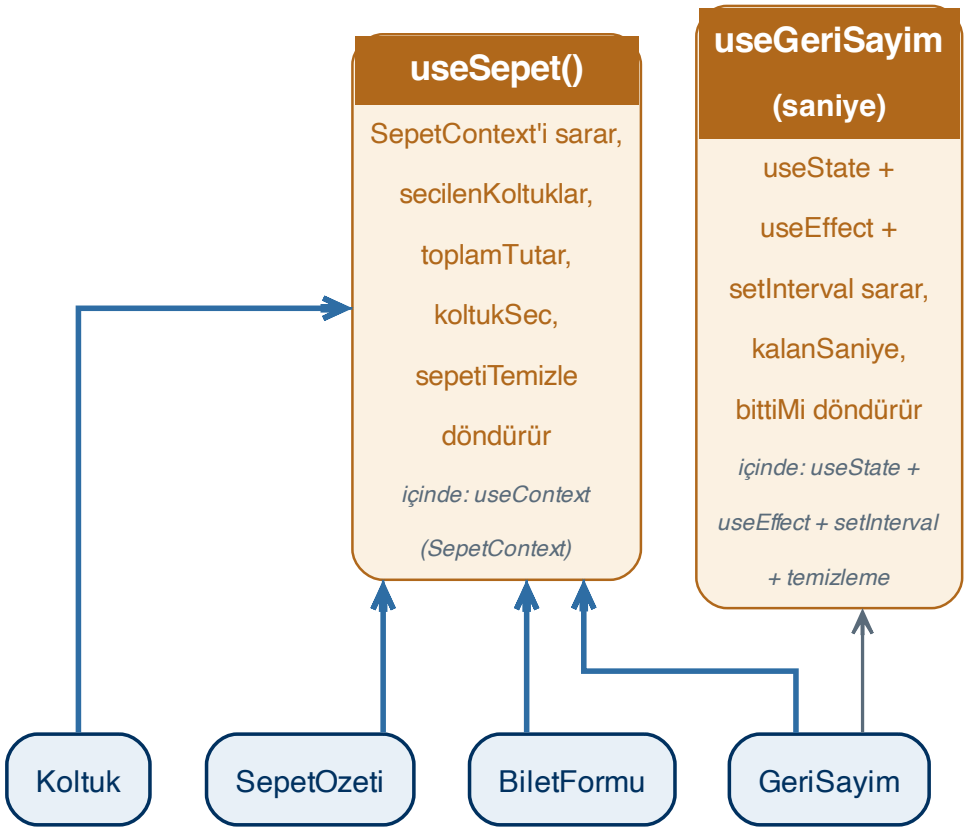
(bkz. Ders notu Hafta 5, Ünite 4 — Özel Hook'lar ve Kompozisyon.)

Adım 8: `SepetOzeti` ve `GeriSayim` bileşenlerini yazın

Ne yapacağız? `src/bilesenler/SepetOzeti.tsx` dosyasını oluşturup `useSepet()`'ten `secilenKoltuklar` ve `toplamTutar`'ı okuyup gösteren küçük bir bileşen yazacaksınız. Ardından `src/bilesenler/GeriSayim.tsx` dosyasını oluşturup `useGeriSayim(120)` çağıran, `bittiMi` olduğunda `useSepet()`'ten gelen `sepetiTemizle()`'yi çağıran ayrı bir `useEffect` kuran ve kalan süreyi `dk:sn` biçiminde gösteren bileşeni yazacaksınız.

Neden? Bölüm 1'de `useGeriSayim`'i doğrudan bir fonksiyonun içine yazmıştınız; şimdi bu fonksiyonu `GeriSayim` bileşeninde yeniden çağırarak, adı `use` ile başlayan, içinde başka hook'lar çağıran bir **özel hook** (*custom hook*) olarak resmen tanıyorsunuz — kopyala-yapıştır yerine aynı mantığı tekrar kullandınız. `useSepet()` de aynı fikrin bir başka uygulamasıdır: `Koltuk`, `SepetOzeti`, `BiletFormu` ve `GeriSayim`'in dördü de kendi başına `useContext(SepetContext)` yazıp `null` kontrolü tekrarlamak yerine yalnızca `const { ... } = useSepet();` yazar — "bir kez yaz, çok yerde kullan" ilkesinin hook'lara uygulanmış hâlidir.

Adım 8 (devamı)



Her bileşen useContext veya useState/useEffect'i KENDİ İÇİNDE TEKRARLAMAZ; ortak mantık hook'ta TEK bir yerde durur.

```jsx
// useSepet()'ten seçili koltuk sayısını ve toplam tutarı okuyup gösteren
// küçük bir bileşen; kendi state'i yoktur, tamamen Context'e dayanır.
export function SepetOzeti() {
  const { secilenKoltuklar, toplamTutar } = useSepet();

  return (
    <div className="sepet-ozeti">
      <p>Seçili koltuk: {secilenKoltuklar.length}</p>
      <p>Toplam tutar: {toplamTutar} TL</p>
    </div>
  );
}
```

```jsx
// useGeriSayim(120) kalan saniyeyi/bittiMi'yi okur; süre bittiğinde
// sepetiTemizle() çağıran ayrı bir efekt kurar — böylece kullanıcı
```

Adım 8 (devamı)

```
// süresi içinde formu göndermezse sepet kendiliğinden boşalır.
const { kalanSaniye, bittiMi } = useGeriSayim(BASLANGIC_SANIYE);
const { sepetiTemizle } = useSepet();

useEffect(() => {
  if (bittiMi) sepetiTemizle();
}, [bittiMi]);
```

```
const dakika = Math.floor(kalanSaniye / 60);
const saniye = kalanSaniye % 60;
const goruntu = `${dakika}:${saniye.toString().padStart(2, '0')}`;

return (
  <div className="geri-sayim">
    {bittiMi ? (
      <p>Süre doldu, sepet temizlendi.</p>
    ) : (
      <p>Sepeti tamamlamak için kalan süre: {goruntu}</p>
    )}
  </div>
);
```

Beklenen sonuç: Dosyalarda hata yok; bu iki bileşen de henüz `App.tsx`'te kullanılmıyor.

İpucu

Bir bileşeni ne zaman ikiye bölmeniz gerektiğine dair pratik sinyaller: (a) dosya okumakta zorlanacak kadar uzuyorsa, (b) aynı JSX/mantık parçası birden fazla yerde tekrarlanıyorsa, (c) bir bileşenin içinde birbiriyle ilgisiz iki sorumluluk (ör. hem form doğrulama hem sepet toplamı hesaplama) varsa. `SepetOzeti`'nin `GeriSayim`'den ayrı bir dosya olması bu üçüncü sinyalin bir örneğidir — ikisi de sepetle ilgilidir ama birbirinden bağımsız işler yapar.

Adım 9: `App.tsx`'i `AppIcerik` ile son hâline getirip tüm bileşenleri birleştirin

Ne yapacağız? `App.tsx`'in içeriğini şu yapıya güncelleyeceksiniz: `App` fonksiyonu artık yalnızca `SepetProvider`'ı (`biletFiyati` prop'uyla) kurar; asıl sayfa içeriği yeni bir `AppIcerik` fonksiyonundadır — `koltuklar/biletler` state'ini tutar, sekme başlığını `secilenKoltuklar.length`'e göre güncelleyen gerçek `useEffect`'i çalıştırır, `BiletFormu`'ndan gelen biletleri `biletleriOlustur` işleviyle işler ve `EtkinlikListesi/KoltukPlani/SepetOzeti/ GeriSayim/BiletFormu`'yu bir arada render eder.

Neden? `AppIcerik`'in `SepetProvider`'ın içinde (bir alt bileşen olarak) durması zorunludur — bir bileşen kendi sağladığı Context'i doğrudan okuyamaz, yalnızca alt'ları okuyabilir; bu yüzden `App`'in kendisi `useSepet()` çağıramaz. Şimdi Bölüm 1'de izole test ettiğiniz belge başlığı efektinin gerçek sürümünü yazıyorsunuz — `secilenKoltuklar`'ın nereden geldiğini artık biliyorsunuz: `useSepet()` aracılığıyla `SepetContext`'ten. `biletleriOlustur`, `BiletFormu`'nun gönderdiği `Bilet[]` dizisini `biletler`'e ekler ve ilgili koltukları `'dolu'` durumuna geçirir — bu, Hafta 4'ün bilinçli olarak yanıtsız bıraktığı "toplam kaç koltuk seçili" sorusunu bugün kapatan son parçadır.

```
// App artık yalnızca hangi etkinliğin bilet fiyatını kullanacağını
```

Adım 9 (devamı)

```
// belirleyip SepetProvider'ı kurar; asıl sayfa içeriği AppIcerik'tedir.
// AppIcerik, SepetProvider'ın İÇİNDE (bir alt bileşen olarak) durmak
// ZORUNDADIR — bir bileşen kendi sağladığı Context'i doğrudan okuyamaz,
// yalnızca alt'ları okuyabilir.
export function App() {
  return (
    <SepetProvider biletFiyati={AKTIF_ETKINLIK.biletFiyati}>
      <AppIcerik />
    </SepetProvider>
  );
}
```

```
// Koltukların GÜNCEL durum listesi (bos/dolu) ve oluşturulan biletler
// burada, App/AppIcerik düzeyinde tutulur (sunucu yok, bu yüzden
// kalıcılık yok — sayfa yenilenince ikisi de kaybolur).
const [koltuklar, setKoltuklar] = useState<KoltukTipi[]>(ilkKoltuklar);
const [biletler, setBiletler] = useState<Bilet[]>([]);
const { secilenKoltuklar } = useSepet();
```

```
// Sekme başlığını seçili koltuk sayısına göre güncelleyen yan etki.
// secilenKoltuklar'ın NEREDEN geldiğini (SepetContext) durum
// kaldırma/Context ünitesinde göreceksiniz — bu efekt yalnızca
// useEffect'in kendisine (biçimine, bağımlılık dizisine) odaklanır.
useEffect(() => {
  document.title =
    secilenKoltuklar.length > 0
      ? `(${secilenKoltuklar.length}) KTÜ Etkinlik ve Bilet Sistemi`
      : 'KTÜ Etkinlik ve Bilet Sistemi';
}, [secilenKoltuklar.length]);
```

```
// BiletFormu'ndan gelen yeni biletleri listeye ekler ve ilgili
// koltukları 'dolu' durumuna geçirir; bu işlev BiletFormu'ya prop
// olarak geçirilir.
function biletleriOlustur(yeniBiletler: Bilet[]) {
  setBiletler((onceki) => [...onceki, ...yeniBiletler]);

  const doluKoltukIdleri = new Set(yeniBiletler.map((bilet) => bilet.koltukId));
  setKoltuklar((onceki) =>
    onceki.map((koltuk) =>
      doluKoltukIdleri.has(koltuk.id) ? { ...koltuk, durum: 'dolu' } : koltuk,
    ),
  );
}
```

```
return (
  <>
    <Bolum baslik="Etkinlikler">
      <EtkinlikListesi etkinlikler={etkinlikler} />
    </Bolum>
    <Bolum baslik="Koltuk Planı">
      <KoltukPlani koltuklar={koltuklar} />
    </Bolum>
    <Bolum baslik="Sepet">
      <SepetOzeti />
      <GeriSayim />
      <BiletFormu onBiletlerOlustu={biletleriOlustur} />
      {biletler.length > 0 && (
```

Adım 9 (devamı)

```
      <p className="bilet-ozet">Oluşturulan bilet sayısı: {biletler.length}</p>
    )}
  </Bolum>
 </>
);
```

```
.sepet-ozeti {
 display: flex;
 gap: 1.5rem;
 padding: 0.75rem 1rem;
 margin: 1rem 0;
 background-color: var(--renk-acikmavi);
 border-radius: 0.5rem;
 font-weight: bold;
}

.geri-sayim {
 margin: 0.5rem 0 1rem;
 color: var(--renk-kehribar);
 font-weight: bold;
}

.bilet-formu {
 display: flex;
 flex-direction: column;
 gap: 0.75rem;
 max-width: 320px;
}

.bilet-formu label {
 display: flex;
 flex-direction: column;
 gap: 0.25rem;
}

.bilet-formu input[type='text'],
.bilet-formu select {
 font: inherit;
 padding: 0.5rem;
 border: 1px solid var(--renk-gri);
 border-radius: 0.25rem;
}

.bilet-formu .hata {
 color: var(--renk-bordo);
 font-weight: bold;
}

.bilet-formu button {
 font: inherit;
 padding: 0.5rem 1rem;
 border: none;
 border-radius: 0.25rem;
 background-color: var(--renk-mavi);
 color: #FFFFFF;
 cursor: pointer;
 align-self: flex-start;
```

Adım 9 (devamı)

```
}

.bilet-ozet {
  color: var(--renk-yesil);
  font-weight: bold;
}
```

Beklenen sonuç: Tarayıcıda üç bölüm görünür: "Etkinlikler" (Hafta 4'ten aynen), "Koltuk Planı" (bir koltuğa tıklamak sepete ekler/çıkarır), "Sepet" (`SepetOzeti`, `GeriSayim` 2:00'den geriye sayar, `BiletFormu`). Bir koltuk seçtiğinizde sekme başlığı `(1) KTÜ Etkinlik ve Bilet Sistemi` biçiminde değişir. Boş ad ile gönderim "Ad alanı boş olamaz." hatası, hiç koltuk seçmeden gönderim "Sepette en az bir koltuk seçili olmalı." hatası gösterir — her iki durumda da `Bilet[]` oluşturulmaz. Ad doldurup en az bir koltuk seçiliyken gönderdiğinizde seçili koltuklar `'dolu'`ya döner, sepet ve form temizlenir, "Oluşturulan bilet sayısı" özeti görünür ve sekme başlığı ilk hâline sıfırlanır.

Beklenen Çıktı

```
Seçili koltuk: 1
Toplam tutar: 150 TL
Sepeti tamamlamak için kalan süre: 2:00
```

Ortak Hata / Tuzak

`App`'in içine yanlışlıkla `SepetProvider`'ı unutup doğrudan `AppIcerik`'i döndürmek, Bölüm 2'de gördüğünüz "`useSepet()` yalnızca `SepetProvider` içinde kullanılabilir" hatasını geri getirir — `App`'in JSX'inin `<SepetProvider>...</SepetProvider>` ile sarılı olduğunu kontrol edin.

Kontrol Noktası — devam etmeden doğrulayın

Tarayıcıda üç bölümü de gördüyseniz, bir koltuk seçtiğinizde sekme başlığının ve `SepetOzeti`'nin güncellendiğini, geri sayımın çalıştığını ve `BiletFormu`'nun hem hata hem başarı durumunu doğru yönettiğini doğruladıysanız, bu haftanın dört ünitesini de uygulamışsınız demektir — Etkinlik/Bilet Sisteminin ilk sepet katmanı tamamlanmıştır.

Adım 10: Kendiniz ekleyin: `SepetOzeti`'ne "Sepeti Temizle" düğmesi ekleyin

Ne yapacağız? `SepetOzeti.tsx`'te `useSepet()`'ten `sepetiTemizle`'yi de okuyacak, sepette en az bir koltuk seçiliyken görünen bir `<button onClick={sepetiTemizle}>Sepeti Temizle</button>` ekleyeceksiniz.

Neden? Bu görev, `useSepet()`'in tek bir dosyada bile kolayca yeni bir yetenek eklemenize nasıl izin verdiğini deneyimlemenizi sağlar — `sepetiTemizle` zaten `SepetContext`'te tanımlıydı, `GeriSayim` bunu süre dolunca çağırıyordu; siz şimdi aynı fonksiyonu kullanıcının kendi isteğiyle de çağırabileceği bir yol ekliyorsunuz — Context'in "birden fazla bileşenin aynı işlevi paylaşması" fikrinin somut bir örneği.

Beklenen sonuç: En az bir koltuk seçtiğinizde "Sepeti Temizle" düğmesi görünür; tıkladığınızda seçili koltuklar sepetten çıkar, sekme başlığı ilk hâline döner ve düğme kaybolur (sepet artık boş).

Adım 10 (devamı)

İpucu

Düğmeyi koşullu göstermek için `secilenKoltuklar.length > 0 && (...)` kalıbını — Hafta 4'ten tanıdık koşullu render desenini — kullanabilirsiniz.

Adım 11: Derlenebilirlik kapısını çalıştırın

Ne yapacağız? `istemci-react` klasöründe sırasıyla `npm ci`, `npx tsc -b` ve `npm run build` komutlarını art arda çalıştıracaksınız.

Neden? Bu üç komut bu haftanın da "kapısıdır" — yazdığınız tüm TypeScript/TSX kodunun (Context, hook'lar, form state'i, bileşenler) tutarlı olduğunu ve projenin gerçek bir sunucuya yüklenebilecek durumda olduğunu garanti eder. `-b` bayrağını unutmayın: kök `tsconfig.json` yalnızca proje referansları içerdiğinden bayraksız `tsc --noEmit` hiçbir şey denetlemez; `npx tsc -b` referans projelerini gerçekten derler (`npm run build`'in ilk adımı da budur). Bir tip hatası varsa zincir `npx tsc -b` adımında durur.

Terminal

```
$ npm ci
$ npx tsc -b
$ npm run build
```

Beklenen sonuç: Üç komut da hatasız biter; son komut `dist/index.html` ve `dist/assets/` altında CSS/JS dosyalarını üretir.

Beklenen Çıktı

```
added 27 packages, and audited 28 packages in 707ms

found 0 vulnerabilities

vite v8.2.2 building client environment for production...
✓ 28 modules transformed.
dist/index.html                   0.47 kB │ gzip:  0.31 kB
dist/assets/index-BgfQDr0S.css    2.45 kB │ gzip:  0.87 kB
dist/assets/index-DsphEfy8.js   196.00 kB │ gzip: 61.95 kB

✓ built in 361ms
```

Sorun Giderme

Belirti	Olası neden	Çözüm
"`useSepet()` yalnızca `SepetProvider` içinde kullanılabilir" hatası	`App`'in JSX'i `SepetProvider` ile sarılmamış ya da bir bileşen `AppIcerik`'in dışında render ediliyor	`App.tsx`'te `<SepetProvider>...</SepetProvider>`'ın `AppIcerik`'i (ya da eski test bileşenini) içine aldığını kontrol edin
Tarayıcı konsolunda "Maximum update depth exceeded" uyarısı, sayfa donuyor	Bir `useEffect` bağımlılık dizisi eksik ya da yanlış, efekt içinde bir `setState` her render'ı yeniden tetikliyor	Efektin gerçekten hangi değerlere bağlı olduğunu (ör. `[secilenKoltuklar.length]`) gözden geçirin, dizinin boş kalmadığından emin olun
`checkbox` her zaman işaretsiz görünüyor, tıklamak hiçbir şey değiştirmiyor	`checked` yerine yanlışlıkla `value` kullanılmış ya da `e.target.checked` yerine `e.target.value` okunmuş	`checkbox`'ta her zaman `checked`/`e.target.checked` çiftini kullandığınızı kontrol edin
Formu gönderince sayfa yeniden yükleniyor, tüm state sıfırlanıyor	`onSubmit` işleyicisinde `e.preventDefault()` eksik ya da en üst satırda değil	`submitEt` fonksiyonunun ilk satırının `e.preventDefault()` olduğunu doğrulayın
Geri sayım hiç durmuyor, bileşeni ekrandan kaldırdıktan sonra bile konsolda uyarı çıkıyor	`useEffect`'in temizleme fonksiyonu (`return () => clearInterval(id)`) yazılmamış	`useGeriSayim.ts`'teki efektin bir fonksiyon döndürdüğünü ve bu fonksiyonun `clearInterval(id)` çağırdığını kontrol edin
TypeScript "Property 'toplamTutar' does not exist on type 'null'" hatası	`useContext(SepetContext)` doğrudan çağrılmış, `useSepet`'in `null` kontrolü atlanmış	`useSepet()`'i kullanın (`useContext` doğrudan değil) — dönüş tipi artık `null` içermez

5 • İzleme — Aktarım — Başka Nerede?

Bugün `Koltuk`'un Hafta 4'te kendi içinde hapis kalan seçimini `SepetContext` üzerinden `App` düzeyine taşıdınız, sekme başlığını ve bir geri sayımı `useEffect` ile güncellediniz, kontrollü bir formla gerçek `Bilet` nesneleri oluşturdunuz ve tekrar eden mantığı `useSepet`/ `useGeriSayim` özel hook'larına taşıdınız — Etkinlik/Bilet Sisteminin ilk gerçek sepet katmanı bugün tamamlandı. Hafta 10'da bu aynı Context kalıbıyla bir `OturumContext` (giriş yapmış kullanıcı bilgisi) kuracaksınız — "birden çok bileşenin ihtiyaç duyduğu ortak durum" sorusunun cevabı bundan sonra her yerde Context olacaktır; bugün kurduğunuz tiplenmiş form state'i + doğrulama kalıbı da Hafta 10'un giriş/kayıt formlarında aynı `value`/ `onChange`/hata-mesajı desenle tekrar karşınıza çıkacaktır. Hafta 6'da bugün `AppIcerik`'in yerel belleğinde (state) yaşayan `Bilet[]` nesneleri — sayfa yenilendiğinde kaybolmayacak biçimde — bir sunucuya (FastAPI) taşınacak; bugün öğrendiğiniz `useEffect` deseni, Hafta 7'de bir API'den gerçek veri çekerken (`fetch`) doğrudan aynı biçimde karşınıza çıkacaktır.